# PLATAFORMA DE SCAN DE VULNERABILIDADES WEB

Prova de Conceito (POC)

ZAP 2.17.0 (Docker) + Nuclei + Amass

FastAPI | Supabase | Lovable | Cloudflare Zero Trust

# PLATAFORMA DE SCAN DE VULNERABILIDADES WEB

Prova de Conceito (POC)

ZAP 2.17.0 (Docker) + Nuclei + Amass

FastAPI | Supabase | Lovable | Cloudflare Zero Trust

| | |
|---|---|
| **Stack** | Lovable + Supabase + FastAPI (Python) + Windows Server 2022 |
| **Infraestrutura** | AWS t3.large + Docker Engine + Cloudflare Tunnel + Zero Trust |

# 1. Visao Geral

Modulo de scan de vulnerabilidades web integrado a plataforma de seguranca em Lovable. Combina ferramentas open source de classe profissional com infraestrutura AWS existente, exposta via Cloudflare Tunnel com Zero Trust — sem nenhuma porta aberta diretamente na instancia.

## 1.1 Objetivo

- Detectar vulnerabilidades web em URLs, dominios e listas de alvos
- Suportar varredura de subdominos via reconhecimento (Amass)
- Retest pontual de vulnerabilidades especificas sem scan completo
- Alertas imediatos para critico e alto — medio/baixo/info registrados silenciosamente
- Relatorios exportaveis em PDF

## 1.2 Engines de Scan

| Engine | Funcao | Tipo | Output |
|---|---|---|---|
| ZAP 2.17.0 (Docker) | DAST comportamental, OWASP Top 10, SQLi, XSS, Client Spider para React/Vue | Ativo | JSON |
| Nuclei | CVEs conhecidos, 9000+ templates, misconfigurations | Ativo | JSON |
| Amass | Reconhecimento de subdominio, enumeracao DNS passiva ou ativa | Passivo/Ativo | JSON |
| Nikto | Headers inseguros, versoes de servidor, paths sensiveis | Ativo | JSON |

# 2. Arquitetura Completa

## 2.1 Fluxo de comunicacao

**Fluxo end-to-end (sem portas abertas na AWS)**

Lovable (React) -> Supabase Edge Function -> Cloudflare Tunnel + Zero Trust -> FastAPI :8000 (Windows Server t3.large) -> ZAP :8080 (Docker) | nuclei.exe | amass.exe -> Supabase DB -> Realtime notifica frontend

## 2.2 Estrutura de pastas na instancia

**Estrutura de diretorios — Windows Server**

```
C:\scanner\
main.py <- FastAPI — endpoints /scan e /retest
normalizers.py <- parsers: normalize_zap() e normalize_nuclei()
retest.py <- retest_zap() e retest_nuclei()
requirements.txt
.env <- variaveis de ambiente (nunca commitar)
C:\scanner\tmp\ <- outputs JSON temporarios dos scans
C:\tools\nuclei\nuclei.exe
C:\tools\amass\amass.exe
C:\tools\nssm\ <- gerenciador de servico Windows
# ZAP roda via Docker container — sem instalacao local do Java
```

## 2.3 Servicos na instancia

| Servico | Como roda | Porta | Gerenciado por |
|---|---|---|---|
| ZAP 2.17.0 | Container Docker (imagem stable — Java 17 embutido) | 127.0.0.1:8080 | Docker --restart always |
| FastAPI | uvicorn main:app --workers 2 | 127.0.0.1:8000 | NSSM (servico Windows) |
| Cloudflared | cloudflared tunnel run | Tunnel HTTPS | NSSM (ja existente) |
| Nuclei | nuclei.exe via subprocess Python | — | FastAPI subprocess |
| Amass | amass.exe via subprocess Python | — | FastAPI subprocess |

# 3. Modulo de Scan — Telas

## 3.1 Dashboard /scans/dashboard

Pagina inicial — visao consolidada do periodo (7, 30 ou 90 dias).

| Scans | Critico | Alto | Medio | Baixo | Resolvidas |
|---|---|---|---|---|---|
| 48 | 7 | 23 | 41 | 18 | 19 |

- Grafico de linha (Recharts) — vulnerabilidades por dia por severity
- Lista das 10 ultimas vulnerabilidades com badge, nome, engine, tempo e botao Retest
- Tabela de scans agendados com frequencia, proxima execucao, botoes Editar e Pausar
- Header: seletor de periodo | Exportar PDF | Novo scan -> /scans/novo

## 3.2 Operacao /scans/novo

| Componente | Detalhe |
|---|---|
| Seletor de escopo (pills) | URL avulsa \| Dominio + subdominos \| Lista em bulk \| Scan completo |
| Input do alvo | Campo URL ou dominio; bulk aceita textarea + importar .txt |
| Cards de engine (toggle multiplo) | ZAP \| Nuclei \| Amass (modo passivo) \| Nikto |
| Opcoes adicionais | Modo passivo Amass (on) \| Seguir redirecionamentos \| Alertar critico/alto (on) |
| Botao Iniciar scan | Insere em scans (status=queued), chama Edge Function, redireciona para /acompanhamento |
| Botao Agendar | Modal: frequencia diaria/semanal/mensal + horario -> insere em scan_schedules |

## 3.3 Acompanhamento /scans/acompanhamento

### Secao 1 — Scans ativos (Realtime em tabela scans)

- Lista scans com status queued ou running, barra de progresso, tempo estimado
- Botao Ver fases: expande inline — recon -> spider -> active scan -> consolidando
- Botao Cancelar: atualiza status para cancelled

### Secao 2 — Alertas imediatos (Realtime WHERE should_alert = TRUE)

- Critico: fundo vermelho suave | Alto: fundo ambar suave
- Cada alerta: badge severity + nome + CVSS + engine + tempo + botao Retest + Ver detalhes
- Medio/baixo/info: registrados silenciosamente — nota fixa na tela

### Secao 3 — Vulnerabilidades do scan atual (lista completa)

- Pills de filtro: severity | status (aberta/resolvida/falso positivo) | engine
- Cards accordion: fechado mostra badge+nome+CVSS+engine+Retest+Falso+
- Expandido adiciona: URL em bloco de codigo, descricao, remediacao, historico de retests

- Retest: spinner durante execucao, badge resultado ao finalizar
- Falso+: modal de confirmacao -> status=false_positive

# 4. Banco de Dados — Supabase

## 4.1 Tabelas

| Tabela | Finalidade | Realtime |
|---|---|---|
| scans | Cada scan disparado — status, progresso, engines, targets | Sim |
| vulnerabilities | Vulnerabilidades com campos normalizados + raw_output JSONB | Sim |
| retest_history | Historico de cada retest por vulnerabilidade | Nao |
| recon_subdomains | Subdominios descobertos pelo Amass | Nao |
| scan_schedules | Agendamentos recorrentes de scans | Nao |

**Por que campos normalizados alem do raw_output?**

O retest precisa saber exatamente qual URL atacar, qual parametro injetar e qual payload usar. O raw_output (JSONB) existe apenas para auditoria. Sem matched_url, param e attack_payload normalizados como colunas separadas, o retest seria um scan completo — inutil como feature.

## 4.2 Schema SQL completo

**Supabase SQL Editor — copiar e executar**

```
-- SCANS
CREATE TABLE scans (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
target_url TEXT,
domain TEXT,
mode TEXT DEFAULT 'url',
engines TEXT[] DEFAULT '{}',
status TEXT DEFAULT 'queued',
progress INT DEFAULT 0,
estimated_seconds_remaining INT,
started_at TIMESTAMPTZ,
finished_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- VULNERABILIDADES
CREATE TABLE vulnerabilities (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
scan_id UUID REFERENCES scans(id) ON DELETE CASCADE,
name TEXT NOT NULL,
severity TEXT,
cvss_score NUMERIC(3,1),
cve_id TEXT,
engine TEXT,
status TEXT DEFAULT 'open',
should_alert BOOLEAN GENERATED ALWAYS AS
```

```
(severity IN ('critical','high')) STORED,
matched_url TEXT,
http_method TEXT,
param TEXT,
attack_payload TEXT,
evidence TEXT,
description TEXT,
remediation TEXT,
raw_output JSONB,
found_at TIMESTAMPTZ DEFAULT NOW()
);
-- RETEST HISTORY
CREATE TABLE retest_history (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
vulnerability_id UUID REFERENCES vulnerabilities(id) ON DELETE CASCADE,
result TEXT NOT NULL,
engine TEXT,
tested_at TIMESTAMPTZ DEFAULT NOW(),
raw_output JSONB
);
-- RECON SUBDOMAINS
CREATE TABLE recon_subdomains (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
scan_id UUID REFERENCES scans(id) ON DELETE CASCADE,
subdomain TEXT NOT NULL,
ip TEXT,
discovered_at TIMESTAMPTZ DEFAULT NOW()
);
-- SCAN SCHEDULES
CREATE TABLE scan_schedules (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
target_url TEXT,
domain TEXT,
mode TEXT,
engines TEXT[],
frequency TEXT,
run_at_time TIME,
next_run_at TIMESTAMPTZ,
active BOOLEAN DEFAULT TRUE,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- INDICES
CREATE INDEX idx_scans_status ON scans(status);
CREATE INDEX idx_scans_created ON scans(created_at DESC);
CREATE INDEX idx_vulns_scan ON vulnerabilities(scan_id);
CREATE INDEX idx_vulns_sev ON vulnerabilities(severity);
CREATE INDEX idx_vulns_alert ON vulnerabilities(should_alert) WHERE should_alert = TRUE;
CREATE INDEX idx_retest_vuln ON retest_history(vulnerability_id);
-- REALTIME
ALTER PUBLICATION supabase_realtime ADD TABLE scans;
ALTER PUBLICATION supabase_realtime ADD TABLE vulnerabilities;
```

# 5. Configuracao da Instancia Windows Server 2022

> **O que instalar manualmente (nao gerado pelo Claude)**
>
> Docker Engine, ZAP container, Nuclei, Amass e NSSM precisam ser instalados manualmente na instancia antes de fazer o deploy do codigo. O codigo Python e as Edge Functions sao gerados pelo Claude com base nesta POC.

## 5.1 Pre-requisitos

- Python 3.12: winget install Python.Python.3.12
- Docker Engine instalado via script oficial Microsoft
- Cloudflared ja configurado com tunnel e Zero Trust ativo
- Subdominio scanner.seudominio.com apontando para a instancia

## 5.2 Instalar Docker Engine

**PowerShell como Administrador**

```
# 1. Habilitar feature de Containers
Install-WindowsFeature -Name Containers -Restart
# 2. Apos reiniciar — instalar Docker Engine (script oficial Microsoft)
$url = 'https://raw.githubusercontent.com/microsoft/Windows-Containers/Main/helpful_tools/Install-DockerCE/install-docker-ce.ps1'
Invoke-WebRequest -UseBasicParsing $url -OutFile install-docker-ce.ps1
.\install-docker-ce.ps1
# 3. Verificar
Get-Service docker
docker version
```

## 5.3 Subir ZAP 2.17.0 via Docker

Imagens oficiais disponiveis em ghcr.io/zaproxy/zaproxy — use sempre stable em producao:

| Variante | Imagem | Quando usar |
|---|---|---|
| stable | ghcr.io/zaproxy/zaproxy:stable | Producao — ZAP 2.17.0 + Java 17 embutido |
| bare | ghcr.io/zaproxy/zaproxy:bare | CI/CD minimalista, sem add-ons extras |
| weekly | ghcr.io/zaproxy/zaproxy:weekly | Testar novas features antes do release |
| nightly | ghcr.io/zaproxy/zaproxy:nightly | Desenvolvimento — pode ter instabilidades |

**PowerShell — ZAP 2.17.0 em modo daemon permanente**

```
# 1. Baixar imagem (Java 17 ja incluso)
docker pull ghcr.io/zaproxy/zaproxy:stable
# 2. Subir container permanente
# --restart always = reinicia apos reboot da instancia
# -p 127.0.0.1:8080 = apenas interno, nunca exposto para fora
docker run -d ^
```

```
--name zap ^
--restart always ^
-p 127.0.0.1:8080:8080 ^
ghcr.io/zaproxy/zaproxy:stable ^
zap.sh -daemon ^
-host 0.0.0.0 -port 8080 ^
-config api.key=SUA_ZAP_KEY ^
-config api.addrs.addr.name=.* ^
-config api.addrs.addr.regex=true
# 3. Verificar (aguardar ~30 segundos para JVM subir)
Start-Sleep 35
Invoke-RestMethod 'http://127.0.0.1:8080/JSON/core/view/version/?apikey=SUA_ZAP_KEY'
# Operacao do dia a dia:
docker ps # status
docker logs zap --tail 50 # logs
docker restart zap # reiniciar
# Atualizar ZAP (apenas 2 comandos):
docker pull ghcr.io/zaproxy/zaproxy:stable
docker restart zap
```

## 5.4 Instalar Nuclei

**PowerShell**

```
New-Item -ItemType Directory -Path C:\tools\nuclei -Force
$url = 'https://github.com/projectdiscovery/nuclei/releases/latest/download/nuclei_3.3.0
_windows_amd64.zip'
Invoke-WebRequest -Uri $url -OutFile nuclei.zip
Expand-Archive nuclei.zip -DestinationPath C:\tools\nuclei\
Remove-Item nuclei.zip
C:\tools\nuclei\nuclei.exe -update-templates -silent
C:\tools\nuclei\nuclei.exe -version
```

## 5.5 Instalar Amass

**PowerShell**

```
New-Item -ItemType Directory -Path C:\tools\amass -Force
$url =
'https://github.com/owasp-amass/amass/releases/latest/download/amass_windows_amd64.zip'
Invoke-WebRequest -Uri $url -OutFile amass.zip
Expand-Archive amass.zip -DestinationPath C:\tools\amass\
Remove-Item amass.zip
C:\tools\amass\amass.exe version
```

## 5.6 Criar estrutura e instalar dependencias Python

**PowerShell**

```
New-Item -ItemType Directory -Path C:\scanner -Force
New-Item -ItemType Directory -Path C:\scanner\tmp -Force
pip install fastapi uvicorn httpx python-dotenv supabase
```

## 5.7 Arquivo .env

**C:\scanner\.env — preencher com seus valores (nunca commitar)**

```
SCANNER_API_SECRET=chave_secreta_forte_aqui
ZAP_API_KEY=sua_zap_key_aqui
SUPABASE_URL=https://xxxxxxxxxxx.supabase.co
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
CF_CLIENT_ID=seu_cloudflare_client_id
CF_CLIENT_SECRET=seu_cloudflare_client_secret
```

## 5.8 Registrar FastAPI como servico Windows (NSSM)

**PowerShell como Administrador**

```
# Instalar NSSM
New-Item -ItemType Directory -Path C:\tools\nssm -Force
Invoke-WebRequest -Uri https://nssm.cc/release/nssm-2.24.zip -OutFile nssm.zip
Expand-Archive nssm.zip -DestinationPath C:\tools\nssm\
Remove-Item nssm.zip
# Registrar FastAPI como servico
$n = 'C:\tools\nssm\nssm-2.24\win64\nssm.exe'
& $n install ScannerAPI python
& $n set ScannerAPI AppParameters '-m uvicorn main:app --host 127.0.0.1 --port 8000
--workers 2'
& $n set ScannerAPI AppDirectory 'C:\scanner'
& $n set ScannerAPI AppEnvironmentExtra 'PYTHONUNBUFFERED=1'
& $n set ScannerAPI Start SERVICE_AUTO_START
& $n start ScannerAPI
# Verificar
Invoke-RestMethod 'http://127.0.0.1:8000/health'
```

## 5.9 Adicionar scanner no Cloudflared

**C:\Users\Administrator\.cloudflared\config.yml**

```
tunnel: SEU-TUNNEL-ID
credentials-file: C:\Users\Administrator\.cloudflared\SEU-TUNNEL-ID.json
ingress:
- hostname: api1.seudominio.com
service: http://localhost:3000
# Adicionar:
- hostname: scanner.seudominio.com
service: http://localhost:8000
- service: http_status:404
```

**Reiniciar cloudflared**

```
Restart-Service cloudflared
```

# 6. Codigo do Scanner Server

## 6.1 requirements.txt

**C:\scanner\requirements.txt**

```
fastapi
uvicorn
httpx
python-dotenv
supabase
```

## 6.2 main.py

**C:\scanner\main.py**

```
import subprocess, json, asyncio, os
from fastapi import FastAPI, Depends, Header, HTTPException
from pydantic import BaseModel
import httpx
from dotenv import load_dotenv
from normalizers import normalize_zap, normalize_nuclei
from retest import retest_zap, retest_nuclei
load_dotenv()
app = FastAPI()
API_SECRET = os.getenv('SCANNER_API_SECRET')
ZAP_KEY = os.getenv('ZAP_API_KEY')
ZAP_URL = 'http://127.0.0.1:8080'
NUCLEI = r'C:\tools\nuclei\nuclei.exe'
AMASS = r'C:\tools\amass\amass.exe'
TMP = r'C:\scanner\tmp'
def verify_key(x_api_key: str = Header(...)):
if x_api_key != API_SECRET:
raise HTTPException(status_code=401)
class ScanRequest(BaseModel):
url: str | None = None
domain: str | None = None
scan_id: str
mode: str = 'url'
engines: list[str] = ['nuclei', 'zap']
passive: bool = True
class RetestRequest(BaseModel):
matched_url: str
param: str | None = None
http_method: str = 'GET'
engine: str
template_id: str | None = None
@app.get('/health')
async def health(): return {'ok': True}
@app.post('/scan')
async def start_scan(req: ScanRequest, _=Depends(verify_key)):
targets, subdomains = [], []
```

```
if req.mode == 'url' and req.url:
targets = [req.url]
elif req.mode in ('domain','full') and req.domain:
if 'amass' in req.engines:
subdomains = await run_amass(req.domain, req.passive)
targets = subdomains or [f'https://{req.domain}']
findings = []
for target in targets:
tasks = []
if 'nuclei' in req.engines: tasks.append(run_nuclei(target))
if 'zap' in req.engines: tasks.append(run_zap(target))
results = await asyncio.gather(*tasks)
for r in results:
if isinstance(r, list): findings.extend(r)
return {'scan_id': req.scan_id, 'findings': findings, 'subdomains': subdomains}
@app.post('/retest')
async def retest(req: RetestRequest, _=Depends(verify_key)):
if req.engine == 'zap':
return await retest_zap(req.matched_url, req.param, req.http_method)
return await retest_nuclei(req.matched_url, req.template_id)
async def run_amass(domain: str, passive: bool) -> list[str]:
out = fr'{TMP}\amass_{domain.replace(".","_")}.json'
flags = [AMASS,'enum','-d',domain,'-json',out,'-timeout','10']
if passive: flags.append('-passive')
proc = await asyncio.create_subprocess_exec(*flags,
stdout=asyncio.subprocess.DEVNULL, stderr=asyncio.subprocess.DEVNULL)
await proc.wait()
subs = set()
try:
for line in open(out):
e = json.loads(line)
if e.get('name'): subs.add(f'https://{e["name"]}')
except: pass
return list(subs)
async def run_nuclei(url: str) -> list:
safe = url.replace('https://','').replace('http://','').replace('/','_')
out = fr'{TMP}\nuclei_{safe}.json'
subprocess.run([NUCLEI,'-u',url,
'-t',r'C:\tools\nuclei\nuclei-templates\http\',
'-severity','critical,high,medium,low,info',
'-json-export',out,'-silent'], capture_output=True, timeout=600)
try:
return [normalize_nuclei(json.loads(l)) for l in open(out) if l.strip()]
except: return []
async def run_zap(url: str) -> list:
async with httpx.AsyncClient(timeout=600) as c:
await c.get(f'{ZAP_URL}/JSON/spider/action/scan/',
params={'apikey':ZAP_KEY,'url':url})
await asyncio.sleep(30)
r = await c.get(f'{ZAP_URL}/JSON/ascan/action/scan/',
params={'apikey':ZAP_KEY,'url':url})
sid = r.json()['scan']
for _ in range(60):
```

```
await asyncio.sleep(10)
s = await c.get(f'{ZAP_URL}/JSON/ascan/view/status/',
params={'apikey':ZAP_KEY,'scanId':sid})
if s.json()['status'] == '100': break
alerts = await c.get(f'{ZAP_URL}/JSON/core/view/alerts/',
params={'apikey':ZAP_KEY,'baseurl':url})
return [normalize_zap(a) for a in alerts.json().get('alerts',[])]
```

## 6.3 normalizers.py

**C:\scanner\normalizers.py**

```
def map_zap_risk(risk: str) -> str:
return {'High':'critical','Medium':'medium',
'Low':'low','Informational':'info'}.get(risk,'info')
def normalize_zap(alert: dict) -> dict:
return {
'name': alert.get('alert'),
'severity': map_zap_risk(alert.get('risk','')),
'matched_url': alert.get('url'),
'http_method': alert.get('method','GET'),
'param': alert.get('param'),
'attack_payload': alert.get('attack'),
'evidence': alert.get('evidence'),
'description': alert.get('description'),
'remediation': alert.get('solution'),
'engine': 'zap',
'raw_output': alert,
}
def normalize_nuclei(f: dict) -> dict:
info = f.get('info', {})
clf = info.get('classification', {})
return {
'name': info.get('name'),
'severity': info.get('severity'),
'cvss_score': clf.get('cvss-score'),
'cve_id': (clf.get('cve-id') or [None])[0],
'matched_url': f.get('matched-at'),
'http_method': 'GET',
'param': None,
'attack_payload': f.get('curl-command'),
'evidence': (f.get('extracted-results') or [''])[0],
'description': info.get('description'),
'remediation': info.get('remediation'),
'engine': 'nuclei',
'raw_output': f,
}
```

## 6.4 retest.py

**C:\scanner\retest.py**

```
import asyncio, subprocess, json, os
```

```
import httpx
ZAP_KEY = os.getenv('ZAP_API_KEY')
ZAP_URL = 'http://127.0.0.1:8080'
NUCLEI = r'C:\tools\nuclei\nuclei.exe'
TMP = r'C:\scanner\tmp'
async def retest_zap(matched_url: str, param: str, method: str) -> dict:
async with httpx.AsyncClient(timeout=120) as c:
r = await c.get(f'{ZAP_URL}/JSON/ascan/action/scan/',
params={'apikey':ZAP_KEY,'url':matched_url,
'recurse':'false','method':method})
sid = r.json()['scan']
for _ in range(30):
await asyncio.sleep(4)
s = await c.get(f'{ZAP_URL}/JSON/ascan/view/status/',
params={'apikey':ZAP_KEY,'scanId':sid})
if s.json()['status'] == '100': break
alerts = await c.get(f'{ZAP_URL}/JSON/core/view/alerts/',
params={'apikey':ZAP_KEY,'baseurl':matched_url})
found = any(a.get('param') == param
for a in alerts.json().get('alerts',[]))
return {'still_vulnerable': found}
async def retest_nuclei(matched_url: str, template_id: str) -> dict:
safe = matched_url.replace('https://','').replace('/','_')
out = fr'{TMP}\retest_{template_id}_{safe}.json'
subprocess.run([NUCLEI,'-u',matched_url,
'-id',template_id,'-json-export',out,'-silent'],
capture_output=True, timeout=90)
try:
lines = [l for l in open(out) if l.strip()]
return {'still_vulnerable': len(lines) > 0}
except:
return {'still_vulnerable': False}
```

# 7. Edge Functions — Supabase

## 7.1 trigger-scan/index.ts

**supabase/functions/trigger-scan/index.ts**

```
import { createClient } from 'https://esm.sh/@supabase/supabase-js@2'
const SCANNER_URL = Deno.env.get('SCANNER_URL')!
const SCANNER_KEY = Deno.env.get('SCANNER_API_SECRET')!
const CF_ID = Deno.env.get('CF_CLIENT_ID')!
const CF_SECRET = Deno.env.get('CF_CLIENT_SECRET')!
const SUPA_URL = Deno.env.get('SUPABASE_URL')!
const SUPA_KEY = Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!
Deno.serve(async (req) => {
const { url, domain, mode, engines, passive, scan_id } = await req.json()
EdgeRuntime.waitUntil(
fetch(`${SCANNER_URL}/scan`, {
method: 'POST',
headers: {
'Content-Type': 'application/json',
'x-api-key': SCANNER_KEY,
'CF-Access-Client-Id': CF_ID,
'CF-Access-Client-Secret': CF_SECRET,
},
body: JSON.stringify({ url, domain, mode, engines, passive, scan_id })
}).then(async (res) => {
const supabase = createClient(SUPA_URL, SUPA_KEY)
const data = await res.json()
if (data.findings?.length > 0)
await supabase.from('vulnerabilities').insert(
data.findings.map((f: any) => ({ ...f, scan_id }))
)
if (data.subdomains?.length > 0)
await supabase.from('recon_subdomains').insert(
data.subdomains.map((s: string) => ({ scan_id, subdomain: s }))
)
await supabase.from('scans')
.update({ status: 'done', finished_at: new Date().toISOString() })
.eq('id', scan_id)
}).catch(async () => {
const supabase = createClient(SUPA_URL, SUPA_KEY)
await supabase.from('scans')
.update({ status: 'error' }).eq('id', scan_id)
})
)
return new Response(JSON.stringify({ status: 'queued' }), {
status: 202, headers: { 'Content-Type': 'application/json' }
})
})
```

## 7.2 retest/index.ts

**supabase/functions/retest/index.ts**

```
import { createClient } from 'https://esm.sh/@supabase/supabase-js@2'
const SCANNER_URL = Deno.env.get('SCANNER_URL')!
const SCANNER_KEY = Deno.env.get('SCANNER_API_SECRET')!
const CF_ID = Deno.env.get('CF_CLIENT_ID')!
const CF_SECRET = Deno.env.get('CF_CLIENT_SECRET')!
const SUPA_URL = Deno.env.get('SUPABASE_URL')!
const SUPA_KEY = Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!
Deno.serve(async (req) => {
const { vulnerability_id } = await req.json()
const supabase = createClient(SUPA_URL, SUPA_KEY)
const { data: vuln } = await supabase
.from('vulnerabilities')
.select('matched_url, http_method, param, engine, raw_output')
.eq('id', vulnerability_id).single()
if (!vuln) return new Response('Not found', { status: 404 })
const res = await fetch(`${SCANNER_URL}/retest`, {
method: 'POST',
headers: {
'Content-Type': 'application/json',
'x-api-key': SCANNER_KEY,
'CF-Access-Client-Id': CF_ID,
'CF-Access-Client-Secret': CF_SECRET,
},
body: JSON.stringify({
matched_url: vuln.matched_url,
param: vuln.param,
http_method: vuln.http_method,
engine: vuln.engine,
template_id: vuln.raw_output?.['template-id'],
})
})
const { still_vulnerable } = await res.json()
await supabase.from('retest_history').insert({
vulnerability_id, engine: vuln.engine,
result: still_vulnerable ? 'still_vulnerable' : 'resolved',
})
if (!still_vulnerable)
await supabase.from('vulnerabilities')
.update({ status: 'resolved' }).eq('id', vulnerability_id)
return new Response(JSON.stringify({ still_vulnerable }), {
status: 200, headers: { 'Content-Type': 'application/json' }
})
})
```

# 8. Logica de Alertas por Severity

| Severity | CVSS | Alerta imediato | Acompanhamento | PDF |
|---|---|---|---|---|
| Critico | 9.0-10.0 | Sim — fundo vermelho | Sim | Sim |
| Alto | 7.0-8.9 | Sim — fundo ambar | Sim | Sim |
| Medio | 4.0-6.9 | Nao | Lista silenciosa | Sim |
| Baixo | 0.1-3.9 | Nao | Lista silenciosa | Sim |
| Info | 0.0 | Nao | Lista silenciosa | Sim |

**Coluna should_alert — gerada automaticamente pelo banco**

BOOLEAN GENERATED ALWAYS AS (severity IN ('critical','high')) STORED. O codigo nunca checa severity como string — apenas consulta should_alert. Elimina bugs por variacao de maiusculas e centraliza a regra no banco.

# 9. Performance — AWS t3.large (8GB RAM / 2 vCPU)

| Cenario | Tempo estimado | RAM consumida | Scans simultaneos |
|---|---|---|---|
| Nuclei apenas (1 URL) | ~45 segundos | ~200MB | 6-8 |
| ZAP apenas (URL simples) | ~3-4 minutos | ~900MB | 2-3 |
| ZAP + Nuclei (1 URL) | ~4-8 minutos | ~1.1GB | 2 |
| Amass passivo (1 dominio) | ~2-5 minutos | ~150MB | 4-5 |
| Scan completo (Amass+ZAP+Nuclei) | ~15-40 min | ~1.3GB | 1-2 |

**Gargalo: ZAP consome ~800MB de RAM (JVM Java no container)**

Para scans ZAP + Nuclei simultaneos com folga, limite a 2 scans completos em paralelo na t3.large. Scans apenas com Nuclei podem rodar 6+ em paralelo. O campo progress na tabela scans deve ser atualizado pelo FastAPI a cada fase concluida.

# 10. Checklist de Implementacao

## 10.1 Instancia (manual)

- [ ] Python 3.12 instalado
- [ ] Docker Engine instalado e servico docker rodando
- [ ] Container ZAP (stable) rodando na porta 127.0.0.1:8080
- [ ] Nuclei instalado em C:\tools\nuclei\ e templates atualizados
- [ ] Amass instalado em C:\tools\amass\
- [ ] NSSM instalado em C:\tools\nssm\
- [ ] FastAPI registrado como servico ScannerAPI e rodando
- [ ] scanner.seudominio.com adicionado no config.yml do cloudflared
- [ ] Cloudflare Zero Trust: Service Token configurado para o scanner

## 10.2 Banco de dados (Supabase)

- [ ] Tabelas criadas: scans, vulnerabilities, retest_history, recon_subdomains, scan_schedules
- [ ] Coluna should_alert como GENERATED ALWAYS AS
- [ ] Realtime habilitado nas tabelas scans e vulnerabilities
- [ ] Todos os indices criados

## 10.3 Scanner server (codigo Python)

- [ ] C:\scanner\requirements.txt criado e pip install executado
- [ ] C:\scanner\.env preenchido com todas as variaveis
- [ ] C:\scanner\main.py com endpoints /health, /scan, /retest
- [ ] C:\scanner\normalizers.py com normalize_zap() e normalize_nuclei()
- [ ] C:\scanner\retest.py com retest_zap() e retest_nuclei()

## 10.4 Edge Functions (Supabase)

- [ ] trigger-scan deployada e testada
- [ ] retest deployada e testada
- [ ] Variaveis de ambiente configuradas no painel Supabase

## 10.5 Frontend (Lovable)

- [ ] /scans/dashboard com metricas, grafico Recharts e agendamentos
- [ ] /scans/novo com seletor de escopo, engines e opcoes
- [ ] /scans/acompanhamento com Realtime, alertas e lista completa de vulns
- [ ] Botao Retest em cada vulnerabilidade chamando Edge Function retest
- [ ] Botao Falso+ com confirmacao e atualizacao de status
- [ ] Filtros por severity, status e engine
- [ ] Exportar PDF chamando Edge Function dedicada